

Fase 1

María Alexandra Jiménez Suárez

Juan José Álvarez Restrepo

Carolina Ramírez Lotero

Natalia Giraldo Morales

Universidad Pontificia Bolivariana

Facultad de Ingeniería

Cesar Augusto López Gallego

10/08/2026

Inventario de hallazgos

ID	Ubicación	Síntoma observado	Principio comprometido	Impacto en el negocio	Severidad y origen
H-01	ServicioProducto.cs - > CargarDesdeArchivo()VerificarStock() VerificarVencimiento() → línea 47 – 118	La clase concentra gestión de productos, persistencia desde archivos, reglas de inventario y emisión de eventos.	SRP	Un cambio en la persistencia, las reglas de alertas o el mecanismo de notificación obliga a modificar la misma clase, aumentando el costo y riesgo del cambio.	Alta — Propio
H-02	ServicioCliente.cs -> AcumularPuntos() Cargar() → línea 36 – 81	La clase combina gestión de clientes, acumulación de puntos, carga desde archivos y emisión de eventos.	SRP	Cambios independientes en persistencia, reglas de puntos o notificaciones pueden afectar una misma clase y aumentar el riesgo de regresión.	Alta — Propio
H-03	Producto.cs -> MostrarInformacion() → línea 29 - 34	La entidad de dominio contiene lógica de presentación mediante Console.WriteLine	SRP	Un cambio de canal de presentación obliga a modificar el modelo de dominio, aumentando el acoplamiento y dificultando la evolución del sistema.	Media/Alta — Propio
H-04	ServicioUsuario.cs -> Cargar() / Login() → línea 27 - 73	La clase administra usuarios, realiza carga desde archivos y coordina la autenticación.	SRP	Cambios en la fuente de datos o en el flujo de autenticación pueden requerir modificar el mismo servicio.	Media — Asistido por IA / corregido
H-05	ServicioProducto.cs / ServicioProducto / Linea 99-107	En la clase no hay forma de cargar un medicamento liquido ni ningún otro desde archivo sin editar el método.	OCP	Cada vez que se quiera agregar un nuevo tipo de producto habría que modificar el método, arriesgando así lo que ya funciona	Medio - Asistido

H-06	Medicamento.cs / MedicamentoCapsula MedicamentoLiquido / Linea 11	Para soportar un nuevo tipo de medicamento hay que tocar la jerarquía	OCP	Se justifica para la empresa un presupuesto de recursos, tiempo y costos para esta jerarquía.	Medio-Asistido refutado
H-07	Producto.cs / MostrarInformacion / Linea 29	Se declara un método virtual que ninguna subclase lo sobrescribe y en ningún punto del programa se invoca.	OCP	El punto de extensión ya existe, pero está muerto, y en vez de crear una subclase que sobrescriba el método para crear un tipo nuevo de producto hay que tocar el switch.	Medio - Asistido
H-08	ServicioProducto.cs / ServicioProducto → línea 23-24	El constructor del servicio de alto nivel crea EventoStockMinimo y EventoVencimiento con new	DIP	Para cambiar como se notifica un stock bajo o un medicamento pronto a vencer, sin reescribir el servicio para ambos casos.	Alta - Propio
H-09	ServicioCliente.cs /ServicioCliente → línea 18-23	El constructor del servicio de alto nivel crea EventosPuntos con new	DIP	Para poder cambiar como se notifica los puntos acumulados se necesitaría editar el servicio.	Media - Propio
H-10	ServicioMovimiento.cs /ServicioMovimiento → línea 18-23	El constructor del servicio de alto nivel crea EventosMovimiento con new	DIP	Para poder cambiar como se hace seguimiento a un movimiento sería necesario cambiar el servicio.	Baja - Propio
H-11	ServicioProducto.cs/CargarDesdeArchivo() → línea 80-86	El servicio consulta el sistema de archivos (File.Exists,	DIP	La lógica de productos queda atada a un formato de	Alta – Asistido por IA

		File.ReadAllLines) para cargar productos		archivo específico, cambiar la fuente de datos obliga a modificar el servicio.	
H-12	ServicioCliente.cs/Cargar() → línea 52-58	El servicio de clientes accede directamente al sistema de archivos para cargar sus datos	DIP	Cualquier cambio en cómo se almacenan los clientes obliga a tocar el mismo servicio que maneja la acumulación de puntos	Media – Asistido por IA
H-13	ServicioUsuario.cs/Cargar() → línea 42-48	El servicio de usuarios lee el archivo de credenciales directamente, sin ninguna capa intermedia	DIP	Cualquier ajuste en cómo se almacenan los usuarios obliga a tocar el mismo servicio que maneja login y credenciales	Alta - Asistido por IA

Análisis de LSP

Jerarquía Persona -> Cliente / Usuario

No se evidencia una violación del principio LSP. Persona define atributos comunes, pero no establece métodos polimórficos cuyo contrato sea modificado por Cliente o Usuario. Las subclases agregan información y comportamiento propio sin alterar el comportamiento de la superclase.

Jerarquía Producto -> Medicamento -> MedicamentoCapsula / MedicamentoLiquido

No se evidencia una violación de LSP. Medicamento, MedicamentoCapsula y MedicamentoLiquido conservan el comportamiento heredado de Producto y agregan atributos específicos sin modificar el contrato de MostrarInformacion().

No se registra un hallazgo LSP en el inventario porque no existe evidencia suficiente de una violación.

Análisis de ISP

Interfaz

IDescuento

No se evidencia una violación del principio ISP. IDescuento declara un único método (CalcularDescuento(decimal precio)), por lo que ningún cliente que la implemente se ve obligado a definir comportamientos que no usa. ServicioDescuento implementa el contrato completo sin métodos vacíos ni forzados.

Interfaz

IServicioNotificacion

No se evidencia una violación del principio ISP. La interfaz declara un único método (`EnviarNotificacion(string mensaje)`), y `ServicioNotificacion` lo implementa en su totalidad sin comportamientos forzados.

No se registra un hallazgo ISP en el inventario porque no existe evidencia de una interfaz que fuerce a una clase a depender de métodos que no utiliza. Lo que sí se identifica, y se deja como observación, ninguna de las dos interfaces se usa en el código. No aparecen inyectadas ni referenciadas en `Program.cs` ni en los servicios de negocio.

Sugerencias de la IA

Propuesta de IA	Decisión del equipo	Argumento
ServicioProducto viola SRP	Aceptado	La clase combina gestión de productos, persistencia, reglas de inventario y eventos, generando razones de cambio independientes. Esta violación fue detectada desde la lectura y análisis previo del código del sistema, por lo que la IA ayudó a confirmar la hipótesis.
ServicioCliente viola SRP	Aceptado	La clase combina gestión, persistencia, reglas de puntos y eventos. Esta violación fue detectada, de igual manera, desde la lectura y análisis previo del código del sistema, por lo que la IA ayudó a confirmar la hipótesis.
ServicioUsuario viola SRP	Corregido	Se acepta la mezcla de responsabilidades como hallazgo ya identificado previamente, pero se aclara que la autenticación está implementada por <code>AspectoAutenticacion</code> ; <code>ServicioUsuario</code> la coordina.
Producto viola SRP por <code>MostrarInformacion()</code>	Aceptado	El modelo de dominio depende directamente de la presentación por consola. La identificación de dicha inconsistencia ocurrió desde el análisis previo del equipo.
<code>Program.cs</code> es una violación de SRP	Refutado/descartado	<code>Program.cs</code> actúa como punto de entrada y orquestador de la aplicación. Su función de coordinar la ejecución de la aplicación no constituye por sí misma evidencia suficiente de una violación de SRP.
ServicioProducto viola OCP	Aceptado	El método no tiene ningún mecanismo (parámetro, mapeo, delegación) que permita reconocer un tipo distinto de producto en el archivo de carga; agregar un nuevo tipo obliga a editar directamente una lógica que ya funciona, en vez de extenderla.
Medicamento Viola OCP	Refutado	Se descarta esta observación: agregar un nuevo tipo de medicamento (por ejemplo, <code>MedicamentoEfervescente</code>) no exige modificar <code>Producto</code> , <code>Medicamento</code> ni las subclases existentes, basta con crear una nueva clase que herede de <code>Medicamento</code> .

Producto Viola OCP	Aceptado	El punto de extensión existe en el diseño (el método virtual) pero está muerto: ninguna subclase lo aprovecha, y Program.cs reimplementa manualmente la presentación de cada producto con un switch. En la práctica, agregar un nuevo tipo de producto obliga a tocar el menú en vez de simplemente sobrescribir el método ya previsto para eso.
ServicioProducto viola DIP por CargarDesdeArchivo(...)	Aceptado	El servicio consulta File.Exists y File.ReadAllLines para leer persistencia desde disco y cargar productos, se acepta ya que no existe ninguna capa que separe esa lectura del resto de la lógica de negocio. Se debe agregar una nueva interfaz encargada de la lectura de los productos.
ServicioCliente viola DIP por Cargar(...)	Aceptado	El constructor del servicio consulta File.Exists y File.ReadAllLines. Se acepta porque no hay ninguna capa que separe la lógica de negocio por lo que cualquier cambio en el origen de datos obliga a editar el servicio.
ServicioUsuario viola DIP por depender de AspectoAutenticacion estatico	Refutado/Descartado	Login() llama directamente a la clase estática AspectoAutenticacion.Login(), sin pasar por ninguna interfaz. Se descarta porque es una regla de negocio fija y sin posibles cambios futuros, forzar una interfaz no aporta valor al sistema.
ServicioUsuario viola DIP por Cargar()	Aceptado	Cargar() llama a File.Exists y File.ReadAllLines directamente dentro del servicio de usuarios. Se acepta, y con más peso que los demás, por tratarse del módulo que maneja credenciales.
Producto viola DIP por usar Console.WriteLine en MostrarInformacion()	Refutado/Descartado	MostrarInformacion() llama a Console.WriteLine directamente desde Producto. Se descarta como violación de DIP porque representa más como una violación de SRP o OCP.

Evidencias

inicio-login

```

PS C:\Users\Leyla\OneDrive\Escritorio\SolucionFarmacia\SolucionFarmacia\AppFarmaciaConsola> dotnet run
Cargando información del sistema...

Productos cargados
Clientes cargados
Usuarios cargados

===== LOGIN =====
Usuario: 

```

Escenario: Inicio del sistema desde consola

Entrada: Ninguna

Comportamiento observado:

- Carga información de productos.
- Carga información de clientes.
- Carga información de usuarios.
- Solicita usuario para iniciar sesión

```

Login correcto
ALERTA: stock mínimo de Dolex
ALERTA: stock mínimo de Amoxicilina
ALERTA: stock mínimo de Loratadina
ALERTA: stock mínimo de Diclofenaco
ALERTA: Dolex próximo a vencer
ALERTA: Ibuprofeno próximo a vencer
ALERTA: Amoxicilina próximo a vencer
ALERTA: Acetaminofen próximo a vencer
ALERTA: Omeprazol próximo a vencer
ALERTA: Loratadina próximo a vencer
ALERTA: VitaminaC próximo a vencer
ALERTA: Diclofenaco próximo a vencer
ALERTA: Aspirina próximo a vencer
ALERTA: Naproxeno próximo a vencer

=====
          SISTEMA FARMACIA
=====

1. Ver productos
2. Ver clientes
3. Buscar producto
4. Registrar venta
5. Acumular puntos
6. Ver alertas
7. Salir

Seleccione opción: 

```

Evidencia del inicio de sesión y carga del estado inicial.

Después de ingresar credenciales válidas, el sistema muestra “Login correcto”, ejecuta automáticamente las verificaciones de stock y vencimiento, genera las alertas correspondientes y posteriormente presenta el menú principal de operaciones.

Entonces, de lo que llevamos se identifica **reglas/comportamientos observados:**

- Un usuario (admin) con credenciales válidas puede acceder.
- Al ingresar, se verifica automáticamente el stock.
- Al ingresar, se verifica automáticamente el vencimiento.
- Las alertas se muestran antes del menú.
- El sistema presenta 7 opciones de operación.


```
Seleccione opción: 1

===== PRODUCTOS =====
Nombre      Stock  Precio
-----
Dolex       2      5000
Ibuprofeno  10      7000
Amoxicilina 1      12000
Acetaminofen 8      4500
Omeprazol   20     15000
Loratadina  4      9000
VitaminaC   15     6000
Diclofenaco 3      11000
Aspirina    9      7500
Naproxeno   7     13500

=====
          SISTEMA FARMACIA
=====
1. Ver productos
2. Ver clientes
3. Buscar producto
4. Registrar venta
5. Acumular puntos
6. Ver alertas
7. Salir

Seleccione opción: █
```

Consulta del listado de productos.

Al seleccionar la opción 1, el sistema muestra el listado de productos registrados, indicando para cada uno su nombre, cantidad disponible en stock y precio. Después de mostrar la información, el sistema retorna al menú principal para permitir seleccionar otra operación.

```
Seleccione opción: 3

Ingrese nombre producto: Dolex

Producto: Dolex
Precio: 5000
Stock: 2
```

Escenario: Buscar producto existente

Entrada: Dolex

Salida: Nombre, precio y stock

Comportamiento: El sistema encuentra el producto y muestra su información

```
Seleccione opción: 3

Ingrese nombre producto: Dole

Producto: Dolex
Precio: 5000
Stock: 2
```

La búsqueda acepta coincidencias parciales del nombre del producto.

Esto sí es un comportamiento que debemos preservar en el rediseño, porque si después alguien cambia la implementación de búsqueda a una comparación exacta, el comportamiento observable cambiaría.

```
Seleccione opción: 3
Ingrese nombre producto: FKIU
Producto no encontrado
```

Con estas 3 capturas de Búsqueda, se logra identificar que la búsqueda permite coincidencias parciales por nombre. Si encuentra una coincidencia, muestra el producto, precio y stock; si no encuentra ninguna, informa que el producto no existe.

Ahora, se estudiará el ítem registrar venta y entender su comportamiento.

```
Seleccione opción: 4
Nombre producto: Dolex
Cantidad: 1
Movimiento registrado: Venta
```

```
===== PRODUCTOS =====
Nombre      Stock  Precio
-----
Dolex       1      5000
Ibuprofeno  10     7000
```

Cuando se empezó con la analización del comportamiento el stock de Dolex estaba en 2, cuando se realizó el registro de venta, se seleccionó la cantidad de 1, y ya para comprobar el comportamiento de seleccionar más de stock se demuestra que:

```
Nombre      Stock  Precio
-----
Dolex       -3     5000
Ibuprofeno  10     7000
```

el sistema permite ventas con stock insuficiente y genera stock negativo.

Ahora, se observa el comportamiento del ítem 5 Acumular puntos,

```
Nombre cliente: carlos
Puntos: 10
Cliente Carlos acumuló 10 puntos
```

Regla de comportamiento identificada : El sistema permite acumular puntos a un cliente existente y suma la cantidad ingresada a los puntos que ya tenía. La búsqueda del cliente no depende de coincidir exactamente en mayúsculas/minúsculas.

Se verificó de nuevo añadiendo más puntos al cliente Carlos, para verificar si solo reemplaza o acumula, y confirmamos que la operación es:

puntos nuevos = puntos anteriores + puntos ingresados

Se realizó una tercera prueba,

Cliente: carlo

Puntos: 6

→ Cliente Carlos acumuló 6 puntos

→ Carlos: 15 → 21

carlo encontró a Carlos.

```
Nombre cliente: carlo
Puntos: 6
Cliente Carlos acumuló 6 puntos
```

Por lo tanto: La búsqueda del cliente permite coincidencias parciales y no exige coincidencia exacta del nombre.

Luego de analizar el comportamiento desde consola, se revisará el código.

1. Buscar producto.

Desde program.cs se utiliza :

```
p.Nombre.ToLower()
.Contains(nombre.ToLower()));
```

Esto confirma exactamente lo que vimos:

- No distingue mayúsculas/minúsculas.
- Permite coincidencias parciales.
- Devuelve el primer producto encontrado.

2. Registrar venta.

Desde program.cs se usa:

```
productoVenta.Stock -=
    cantidad;

Movimiento venta =
    new Movimiento(
        DateTime.Now,
        cantidad,
        "Venta",
        productoVenta);

servicioMovimiento
    .RegistrarMovimiento(
        venta);

Console.WriteLine(
    "\nVenta registrada");
```

Se identifica el comportamiento de que el sistema registra la venta sin validar que la cantidad solicitada sea menor o igual al stock disponible. Por eso permite stock negativo.

3. Acumular puntos.

Los puntos nuevos se suman a los puntos existentes.

La búsqueda del cliente permite coincidencias parciales y no distingue mayúsculas/minúsculas.

4. Alertas de stock

```
productoVenta.Stock -=  
    cantidad;
```

Con este comportamiento se identifica que **Stock nuevo = Stock actual – cantidad vendida**

Por eso comprobamos en las pruebas realizadas al inicio:

Stock inicial: 1

Cantidad vendida: 4

Stock final: -3

Entonces, si la cantidad vendida supera el stock disponible, el sistema permite la operación y genera un stock negativo.

Y Eso está directamente respaldado por el código y por nuestra prueba realizada anteriormente.

El objeto Movimiento se crea después:

```
Movimiento venta =  
    new Movimiento(  
        DateTime.Now,  
        cantidad,  
        "Venta",  
        productoVenta);
```

Y luego:

```
servicioMovimiento  
    .RegistrarMovimiento(  
        venta);
```

Por eso podemos distinguir dos comportamientos:

- **Cambio de estado:** se reduce el stock.
- **Registro del movimiento:** se crea/registra una venta.

Desde el archivo ServicioProducto.cs , esta la regla exacta, ya que se observa el comportamiento de que el sistema genera una alerta cuando el stock de un producto es menor o igual a su stock mínimo configurado.

```

public void VerificarStock()
{
    foreach (var producto in productos)
    {
        if (producto.Stock <=
            producto.StockMinimo)
        {
            EventoStock.Disparar(producto);
        }
    }
}

```

Regla de comportamiento

Si:

Stock actual $\leq$ Stock mínimo

Entonces:

Se genera la alerta de stock mínimo.

El $\leq$ significa que cuando el stock es exactamente igual al mínimo también genera alerta.

5. Alertas de vencimiento

```

// ===== ALERTAS =====

servicioProducto.VerificarStock();

servicioProducto.VerificarVencimiento();

servicioProducto.EventoVencimiento.Vencimiento +=
mensaje =>
{
    Console.ForegroundColor =
        ConsoleColor.Yellow;

    Console.WriteLine(mensaje);

    Console.ResetColor();
};

```

Desde program.cs se ejecuta la verificación de vencimiento y, si se genera un evento, muestra el mensaje en amarillo, entonces el comportamiento entendido es el sistema ejecuta una verificación de fechas de vencimiento y, cuando se genera una alerta, esta es comunicada mediante el evento Vencimiento y mostrada al usuario.

```

public void verificarVencimiento()
{
    foreach (var producto in productos)
    {
        int dias =
            (producto.FechaVencimiento -
             DateTime.Now).Days;

        if (dias <= 30)
        {
            EventoVencimiento
                .Disparar(producto);
        }
    }
}

```

Desde ServicioProducto.cs,

también se observa el comportamiento de que el sistema genera una alerta cuando faltan 30 días o menos para la fecha de vencimiento del producto.

Regla de comportamiento

Si: días hasta vencimiento $\leq$ 30

Entonces: Se genera una alerta de vencimiento. Y nuevamente tenemos un caso límite interesante: Exactamente 30 días antes también genera alerta.

Puntos de dolor priorizados

Dolor #1: ServicioProducto.cs concentra demasiadas responsabilidades

Este viene siendo relacionado con **H-01**.

Actualmente ServicioProducto mezcla:

- gestión de productos;
- lectura desde archivos;
- reglas de stock;
- reglas de vencimiento;

- generación de eventos.

Esto hace que diferentes tipos de cambios terminen afectando la misma clase. Por ejemplo, si se quisiera cambiar la forma de almacenar los productos o modificar la lógica de las alertas, sería necesario intervenir ServicioProducto. y un error de vencimiento mal detectado puede traducirse en medicamento vencido vendido o desperdiciado (pérdida directa + riesgo legal/regulatorio en el sector salud).

Impacto: Cada vez que alguien toque esa clase, para cambiar cómo se guarda un archivo, cómo se calcula el stock mínimo o cómo se avisa un vencimiento, tiene que volver a probar TODO lo demás, porque están enredados.

Lo consideramos el problema más importante porque concentra varias responsabilidades relacionadas con diferentes partes del sistema y, por lo tanto, un cambio puede tener un impacto mayor y aumentar el riesgo de afectar comportamientos que actualmente funcionan.

Relacionados: H-01, H-08 y H-11.

Dolor #2 : Los servicios dependen directamente de los archivos de texto

Encontramos que ServicioProducto, ServicioCliente y ServicioUsuario acceden directamente al sistema de archivos mediante métodos como File.Exists() y File.ReadAllLines().

Esto significa que los servicios que contienen lógica del sistema también conocen directamente cómo se almacenan los datos. Actualmente funciona porque el sistema utiliza archivos .txt, pero si en el futuro se quisiera utilizar una base de datos, una API u otra fuente de información, habría que modificar estos servicios.

Lo ubicamos como segundo punto de dolor porque afecta varias partes del sistema y puede hacer más costoso implementar cambios relacionados con la persistencia. Sin embargo, lo consideramos ligeramente menos prioritario que el primer punto porque el problema de ServicioProducto reúne más responsabilidades diferentes dentro de una misma clase.

Relacionados: H-11, H-12 y H-13.

3. La incorporación de nuevos tipos de productos

El sistema ya tiene una estructura de productos y medicamentos, pero encontramos que la incorporación de nuevos tipos no está completamente desacoplada de la lógica existente. Porque, existen decisiones de tipo switch y lógica que debe modificarse cuando se quiere manejar una nueva clasificación de producto.

Esto se vuelve importante teniendo en cuenta las solicitudes futuras de la farmacia, ya que se planea vender no solamente medicamentos, sino también cosméticos, alimentos, inyectología y otros productos.

El problema no significa que actualmente no se puedan agregar nuevos productos, sino que la forma en que está construido el sistema puede hacer que estos cambios requieran modificar código que ya funciona. Esto aumenta el riesgo de introducir errores al incorporar nuevas funcionalidades.

Relacionados: H-05 y H-07.